Universidade Estadual de Santa Cruz (UESC)

Etapa 2 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação sequencial e paralela em memória distribuída com MPI

Relatório acadêmico — DEC107 — Processamento Paralelo

Autores: Gabriel Oliveira Lucas César — Matrícula: 202410123;
Ronaldo Ribeiro Porto Filho — Matrícula: 202410131
Disciplina: DEC107 — Processamento Paralelo
Data: 29/10/2026

---

1 Introdução

Este relatório apresenta a Etapa 2 do projeto de geração do conjunto de Mandelbrot pelo método *escape-time*, agora com processos MPI e memória distribuída. A proposta é comparar uma implementação MPI independente com o código sequencial original da Etapa 1, estudar duas formas estáticas de decomposição da imagem e medir o efeito do número de processos, da granularidade das tarefas e da coleta dos resultados sobre o desempenho.

Para cada pixel, um número complexo c é determinado por interpolação linear das coordenadas da imagem. A partir de $z_0 = 0$, aplica-se

$$z_{n+1} = z_n^2 + c, \qquad \text{escape quando } |z_{n+1}|^2 > 4, \tag{1}$$

até o limite `MAX_ITER`. O número de iterações executadas é salvo numa matriz de inteiros de 32 bits. Embora os pixels sejam independentes, o esforço computacional é heterogêneo: muitos escapam rapidamente e outros exigem centenas ou milhares de iterações. Consequentemente, distribuir quantidades iguais de linhas pode não distribuir quantidades iguais de trabalho.

Diferentemente do OpenMP, no qual *threads* compartilham o mesmo espaço de endereçamento, os processos MPI possuem buffers próprios e precisam trocar dados explicitamente. A distribuição de linhas e a coleta da matriz final passam a integrar o custo de execução. Foram implementadas *faixas contíguas* e *distribuição cíclica de blocos de linhas*; ambas usam atribuição **estática**, e não despacho dinâmico mestre–trabalhador.

2 Metodologia

2.1 Implementações e particionamento

O código `mandelbrot_sequencial.c` da Etapa 1 foi preservado sem alterações. O programa `mandelbrot_mpi.c` implementa o mesmo núcleo *escape-time* com precisão `double`, escrita de matriz `int32` em ordem de linhas e escolha da distribuição por parâmetros de execução. Cada processo computa apenas suas linhas atribuídas, sem compartilhamento de endereços.

Blocos contíguos: as H linhas são divididas em p faixas consecutivas, diferindo no máximo uma linha em quantidade. O *rank* r calcula sua faixa, mas pode receber uma região mais custosa do fractal.

Cíclica com *chunk*: blocos consecutivos de c linhas são atribuídos alternadamente entre os processos pela regra

$$\text{rank}(y) = \left\lfloor \frac{y}{c} \right\rfloor \bmod p, \qquad c \in \{1, 4, 16\}. \tag{2}$$

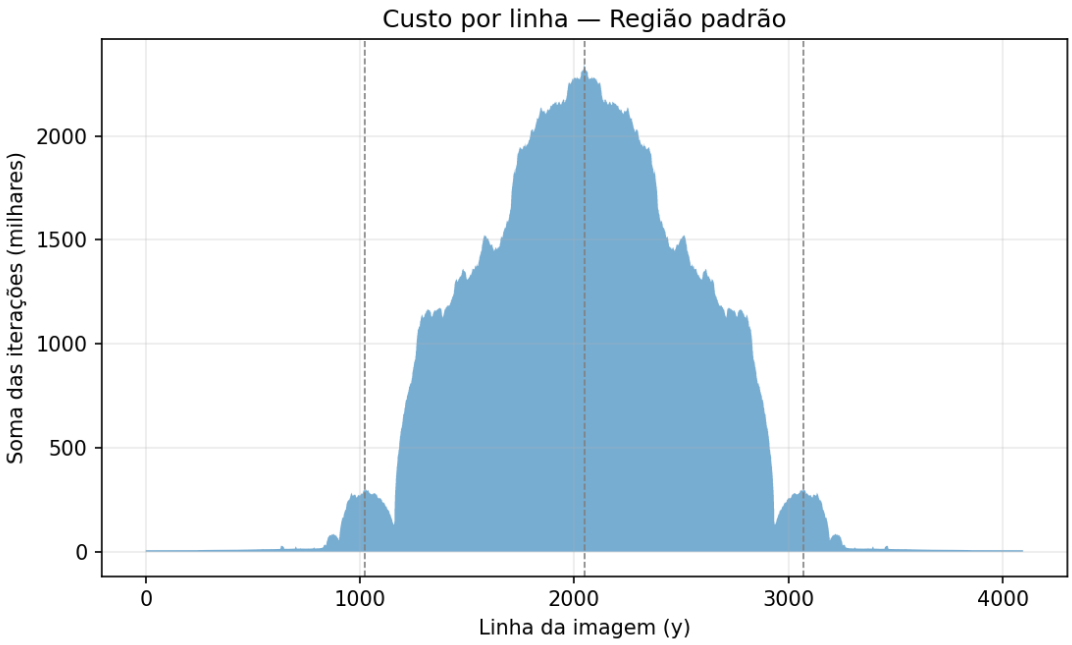


Figura 1: Trabalho computacional agregado por linha na região padrão; sua concentração no centro explica o risco de desbalanceamento por faixas contíguas.

A estratégia espalha partes distantes da imagem entre os processos. O particionamento é definido previamente: não há fila dinâmica de tarefas, nem entrega de trabalho sob demanda.

2.2 Operações MPI e composição da imagem

O programa utiliza `MPI_Init`, `MPI_Comm_rank` e `MPI_Comm_size` para inicialização e identificação dos processos; `MPI_Bcast` para distribuição de parâmetros; `MPI_Allreduce` para detecção coletiva de falhas de alocação; e `MPI_Barrier` para alinhamento do início e encerramento da janela medida. Cada processo produz uma parte da matriz em buffer próprio.

A coleta ocorre por `MPI_Gatherv`, adequado porque a quantidade de linhas por processo pode variar. Para faixas contíguas, os dados são recebidos diretamente na ordem da imagem. Na estratégia cíclica, o processo raiz recebe os buffers agrupados por *rank* e depois reorganiza as linhas nas posições globais, com tempo de montagem medido separadamente. `MPI_Gather` reúne métricas locais e contagens de trabalho, enquanto `MPI_Reduce` obtém a maior duração local. A saída binária é escrita somente pelo processo raiz, após a janela principal de cálculo e coleta.

Não se implementou uma terceira estratégia dinâmica nem um mestre dedicado exclusivamente ao despacho: o *rank* zero também calcula pixels. Assim, a "sobrecarga do mestre" corresponde aqui a coleta, reorganização da matriz quando cíclica, agregação de métricas e escrita, não a gerenciamento dinâmico de tarefas.

2.3 Casos, ambiente e protocolo experimental

Foram avaliados dois casos de escalabilidade forte, ambos em 4096×4096: a região padrão ($\mathrm{Re} \in [-2,1]$, $\mathrm{Im} \in [-1{,}5,1{,}5]$, `MAX_ITER=1000`) e o zoom no vale dos cavalos-marinhos (centro $-0{,}743643887 + 0{,}131825904i$, largura 3×10^{-3} e `MAX_ITER=5000`). Para ambos, foram usados $p \in \{1,2,4,6,8,12\}$ processos, nas quatro configurações: blocos, cíclica com *chunk* 1, 4 e 16.

A escalabilidade fraca foi investigada na região padrão, com resolução aproximada $4096\sqrt{p}$ em cada dimensão: 4096^2, 5793^2, 8192^2, 10033^2, 11585^2 e 14189^2, respectivamente para $p = 1, 2, 4, 6, 8, 12$. Isso mantém aproximadamente 4096^2 pixels por processo, mas não garante o mesmo número de iterações por processo.

Tabela 1: Ambiente de execução da Etapa 2.

Item	Configuração
CPU	Intel Core i5-12400F, 6 núcleos físicos, 12 processadores lógicos
Memória	32 GB de RAM
Sistema	Windows 11
Sequencial	GCC/MinGW 6.3.0 (32 bits), `-O2 -fopenmp -ffp-contract=off -msse2 -mfpmath=sse`
MPI	Microsoft Visual C++ x64 (MSVC), `/O2 /TC /fp:precise`, MS-MPI runtime/SDK
Execução MPI	`mpiexec -n N` em um único computador
Condições manuais	Coleta com aplicativos de segundo plano fechados e sincronização do OneDrive pausada; plano de energia Equilibrado previsto no protocolo. Temperatura e afinidade dos processos não foram registradas.

A bateria oficial totalizou **386 execuções válidas**, com 5 medições por configuração MPI, 7 referências sequenciais no início e 3 no fim de cada caso forte e referências sequenciais específicas para cada tamanho da escalabilidade fraca. As configurações MPI foram embaralhadas dentro de cada rodada; o resultado consolidado foi resumido pela **mediana**, com mínimo, máximo e desvio-padrão preservados nos CSVs. A geração de imagens e a escrita binária foram separadas do tempo principal.

A medição MPI inicia após uma barreira e abrange cálculo, `MPI_Gatherv`, reorganização no raiz quando aplicável e barreira final. O cálculo local e a duração local da coleta são medidos com `MPI_Wtime`; o código sequencial preserva o cronômetro `omp_get_wtime` da Etapa 1. A duração de `MPI_Gatherv` **inclui espera/sincronização** e não pode ser interpretada diretamente como tempo de transferência na rede. Os testes foram locais, com MPI entre processos na mesma máquina.

2.4 Métricas

O tempo de referência T_{seq} é a mediana sequencial correspondente a cada caso e resolução. Para a escalabilidade forte:

$$S(p) = \frac{T_{seq}}{T_{MPI}(p)}, \qquad E(p) = \frac{S(p)}{p} \times 100\%. \tag{3}$$

O fator de balanceamento de carga (FBC), tal como definido no modelo da disciplina, utiliza **somente o tempo de computação de cada processo**, excluindo comunicação e escrita:

$$\mathrm{FBC} = \frac{T^{calc}_{\max} - T^{calc}_{\min}}{T^{calc}_{\max}}, \quad 0 \le \mathrm{FBC} \le 1. \tag{4}$$

Valores próximos de zero indicam tempos de cálculo parecidos; próximos de um indicam grande disparidade. Para escalabilidade fraca foi adotada $E_{fraca}(p) = 100\, T_{MPI}(1, N_1)/T_{MPI}(p, N_p)$, comparando a **mesma estratégia de distribuição** em diferentes resoluções. Ela é uma eficiência temporal relativa, não a eficiência forte S/p.

3 Resultados

3.1 Corretude

Todas as **386 execuções oficiais** apresentaram verificações internas de `hash_ok` e `config_ok` válidas. Para cada combinação de região e resolução, as matrizes binárias MPI tiveram SHA-256 idêntico ao sequencial original. Na comparação explícita das matrizes preservadas de 4096^2, tanto blocos quanto cíclica resultaram em **zero pixels diferentes** em relação à versão sequencial.

A comparação *externa* com a função Python do professor **não foi aprovada integralmente**. A referência Python registra índice de escape i, enquanto o C registra $i + 1$; essa convenção foi ajustada apenas na comparação diagnóstica, sem alterar arquivos ou medições. Mesmo após o ajuste, restaram 786 pixels divergentes (0,004685%, diferença máxima 460) no padrão e 24.242 (0,144494%, diferença máxima 1.415) no zoom. Os limites informados pelo validador da Etapa 1 são 0,01% e diferença máxima de uma iteração; portanto, as duas regiões ficaram fora dos critérios completos. A divergência externa também foi observada na Etapa 1 e **não foi introduzida pelo MPI**.

Tabela 2: Corretude: comparação diagnóstica após ajuste de convenção de contagem.

Caso	Pixels divergentes	Percentual	Diferença máxima
Padrão	786	0,004685%	460
Zoom	24.242	0,144494%	1.415

3.2 Escalabilidade forte: tempo, speedup e eficiência

Para 4096^2, a referência sequencial mediana foi de **6,9825 s** na região padrão e **3,0890 s** no zoom. A Tabela 3 apresenta os melhores tempos entre as quatro configurações por quantidade de processos, sem omitir o contraste de blocos e cíclica em 12 processos.

Tabela 3: Tempos medianos oficiais (s), 4096^2. Cada célula MPI representa 5 repetições.

Processos	Padrão blocos	Padrão cíclica c=16	Zoom blocos	Zoom cíclica c=4
1	7,278	7,362	3,263	3,288
2	3,784	3,822	1,991	1,748
4	3,667	2,010	1,093	0,929
6	3,141	1,398	0,828	0,644
8	2,641	1,084	0,607	0,503
12	1,988	**0,804**	0,489	**0,430**

Na região padrão, com 12 processos e *chunk* 16, o MPI alcançou **speedup 8,682** e **eficiência 72,35%**; por blocos, obteve speedup 3,512 e eficiência 29,26%. No zoom, com 12 processos e *chunk* 4, obteve speedup **7,183** e eficiência **59,86%**; por blocos, speedup 6,318 e eficiência 52,65%. Esses valores usam o baseline da **própria coleta MPI**, não o tempo sequencial da Etapa 1.

3.3 Balanceamento de carga e trabalho por processo

O custo por linha da região padrão é concentrado em regiões específicas, tornando inadequada a suposição de que faixas contíguas de tamanho semelhante exigem tempos parecidos. Em 12 processos, o FBC mediano de blocos foi de **0,995** na região padrão; a distribuição cíclica com *chunk* 16 apresentou FBC de **0,073**. No zoom, os valores correspondentes foram **0,643** para blocos e **0,158** para cíclica com *chunk* 4.

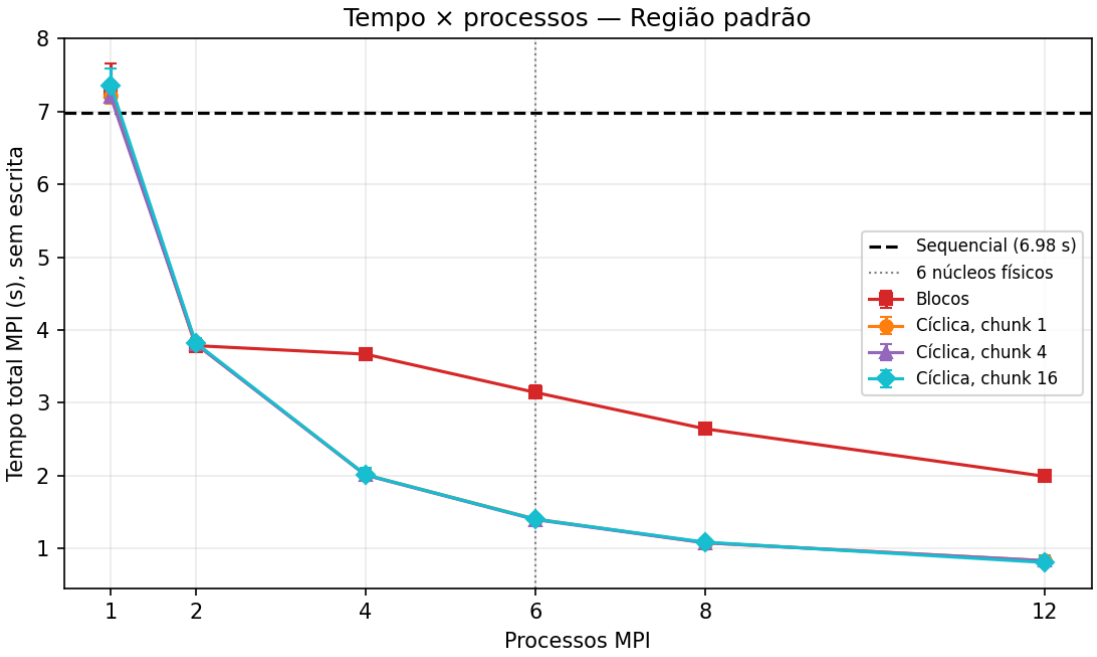


Figura 2: Escalabilidade forte: tempo mediano MPI na região padrão, com barra de variação entre mínimo e máximo e referência sequencial.

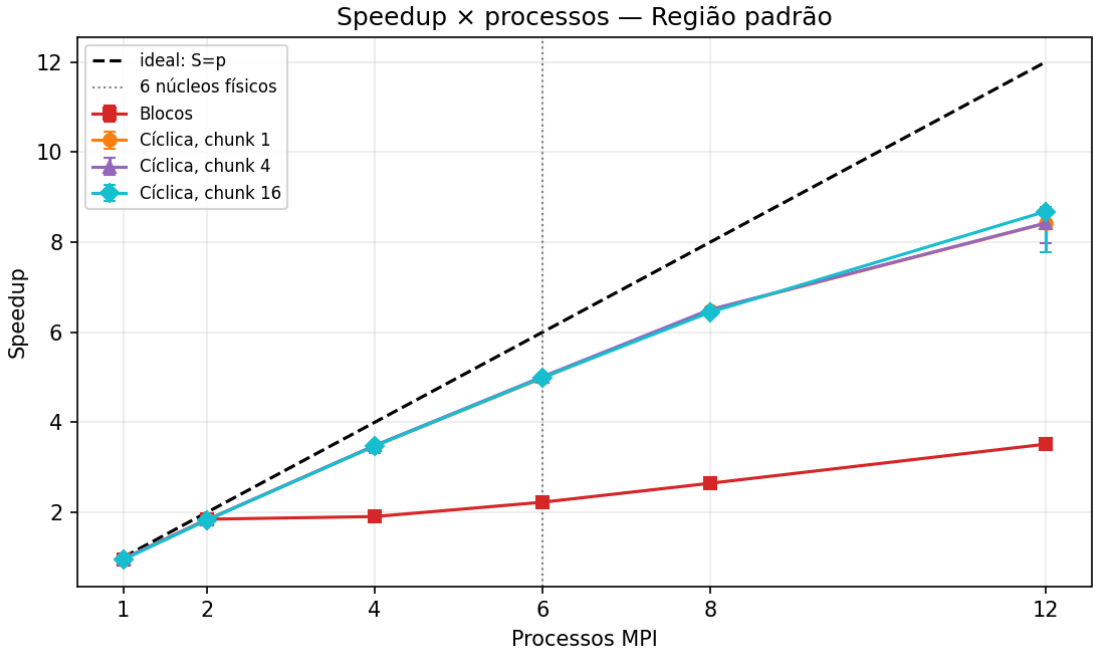


Figura 3: Speedup na região padrão por estratégia, com linha ideal $S(p) = p$.

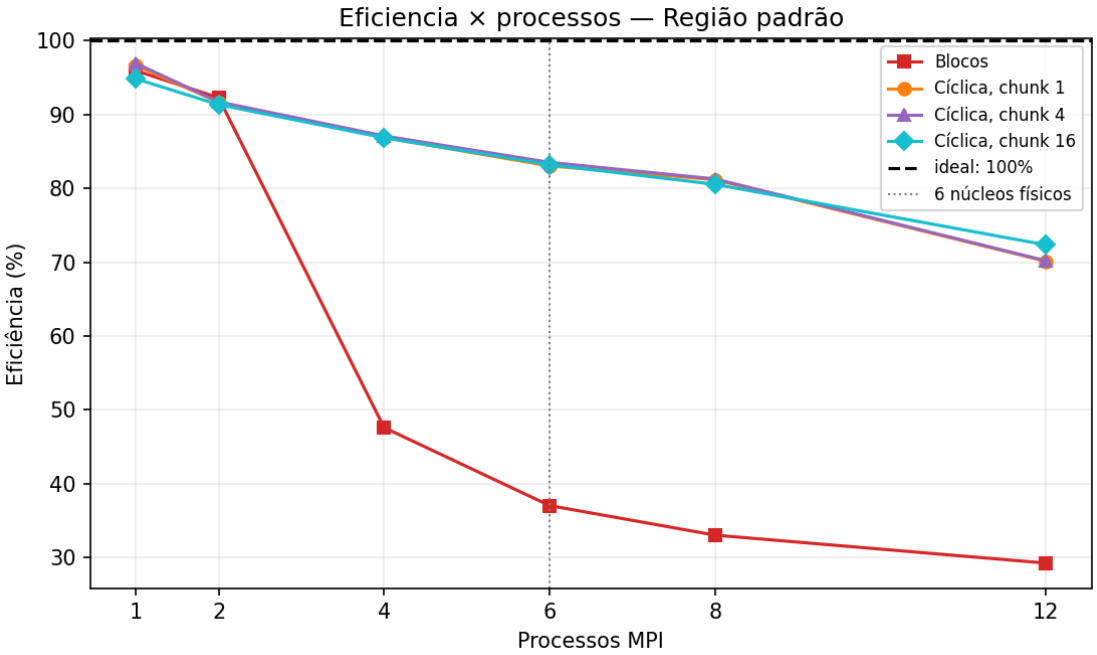


Figura 4: Eficiência paralela na região padrão; a linha vertical destaca os seis núcleos físicos.

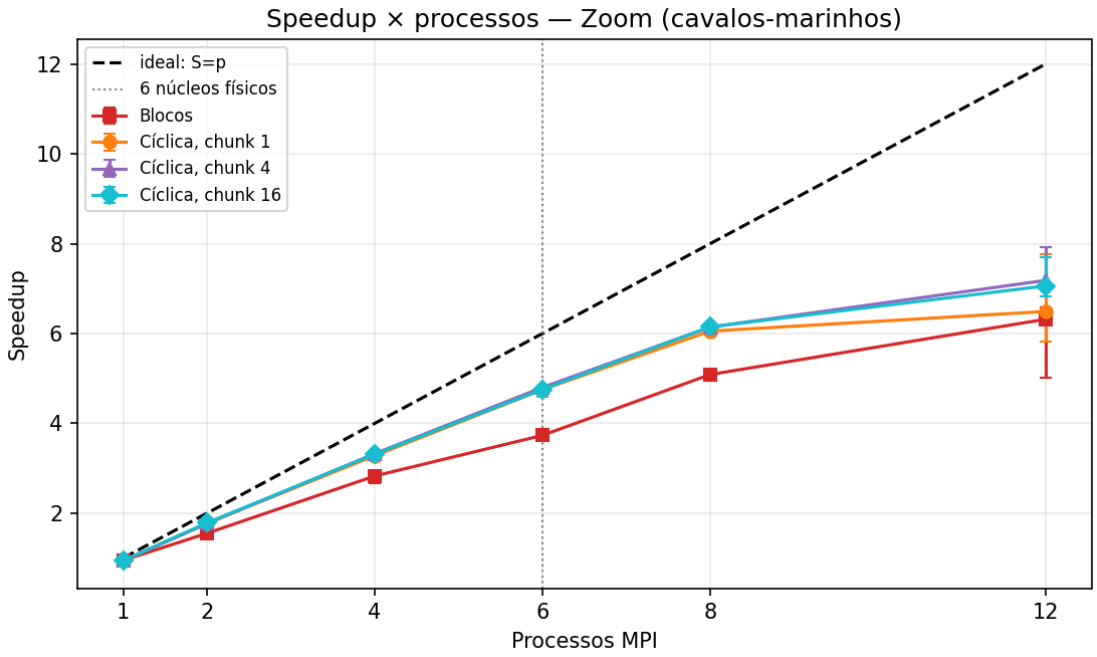


Figura 5: Speedup no zoom do vale dos cavalos-marinhos.

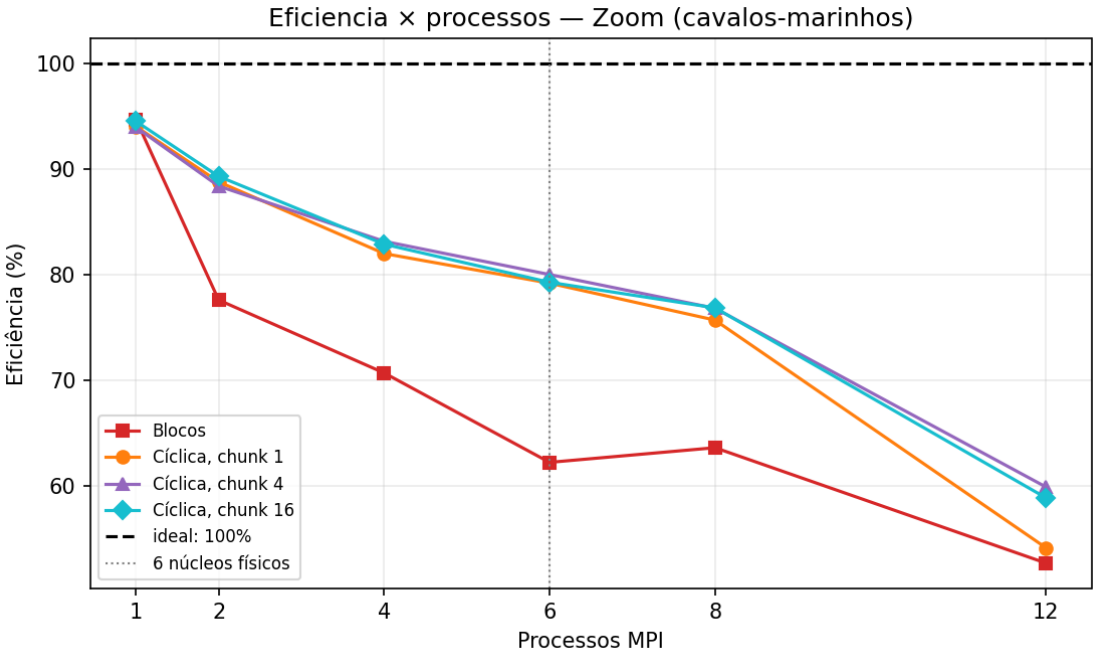


Figura 6: Eficiência paralela no zoom, em função do número de processos.

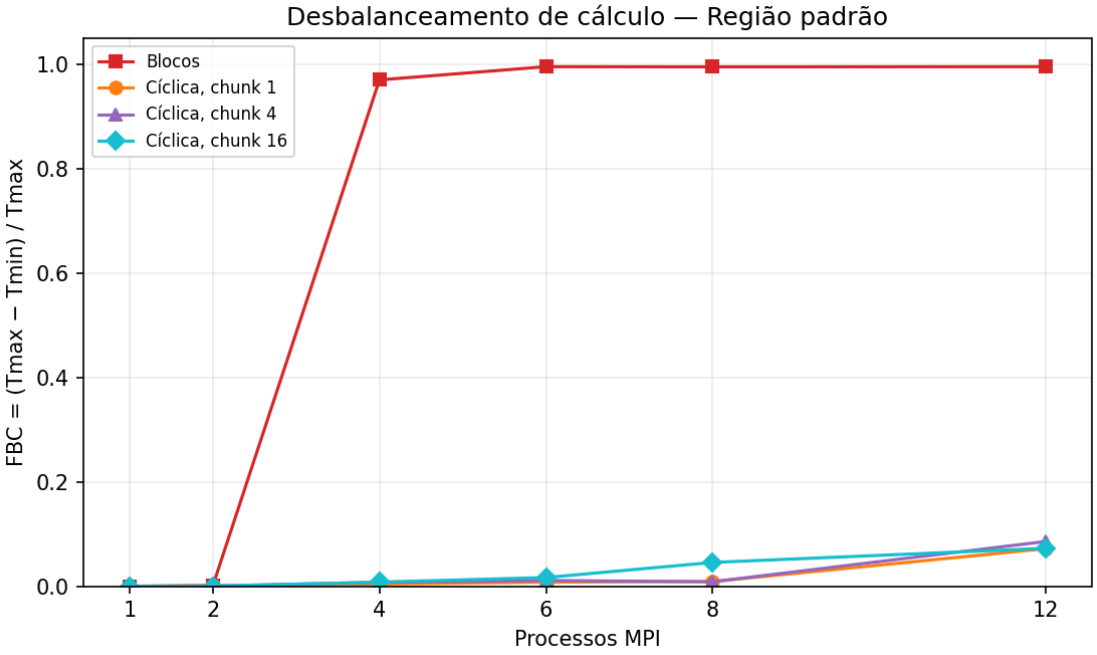


Figura 7: FBC medido a partir dos tempos locais de cálculo: região padrão.

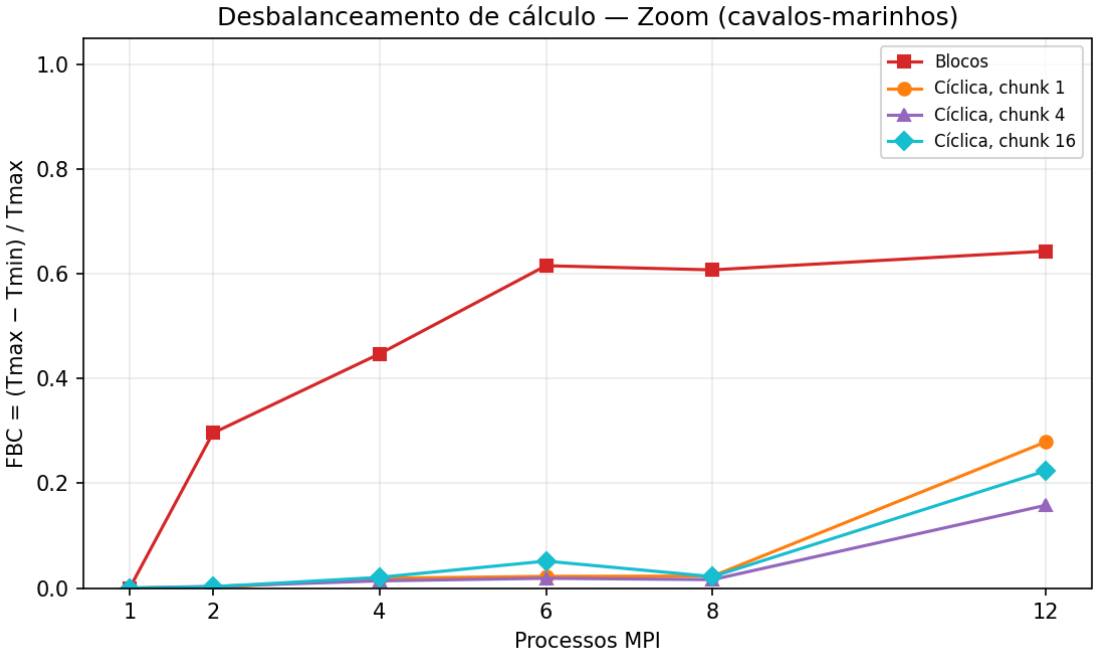


Figura 8: FBC por estratégia no zoom, onde o padrão de trabalho por linha é diferente.

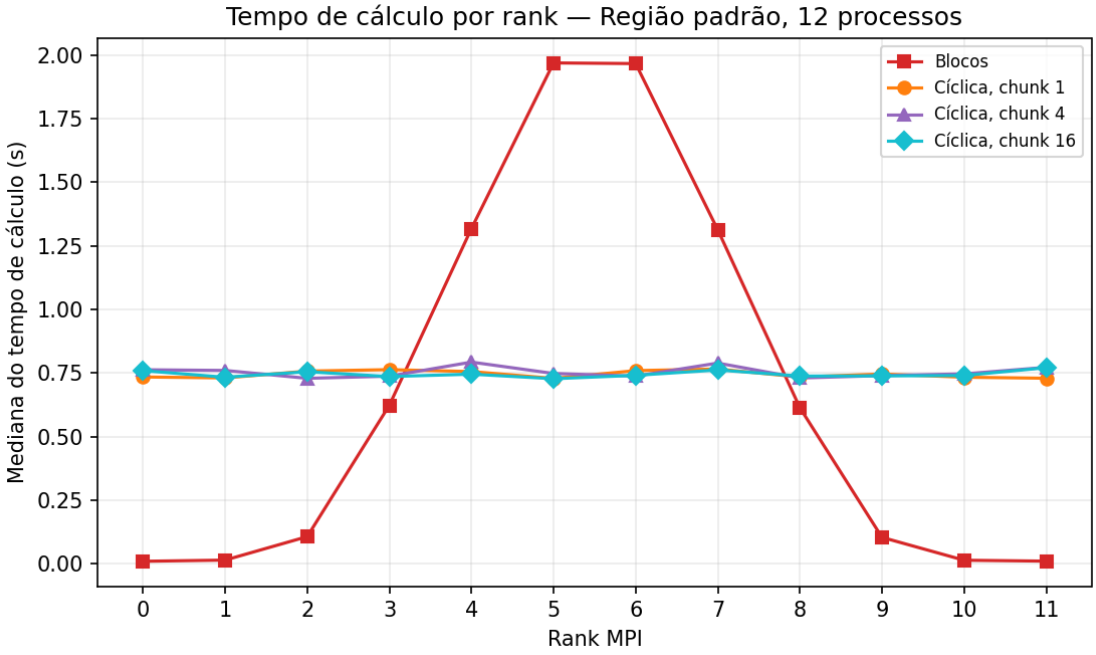


Figura 9: Distribuição do tempo de cálculo entre os 12 ranks no caso padrão.

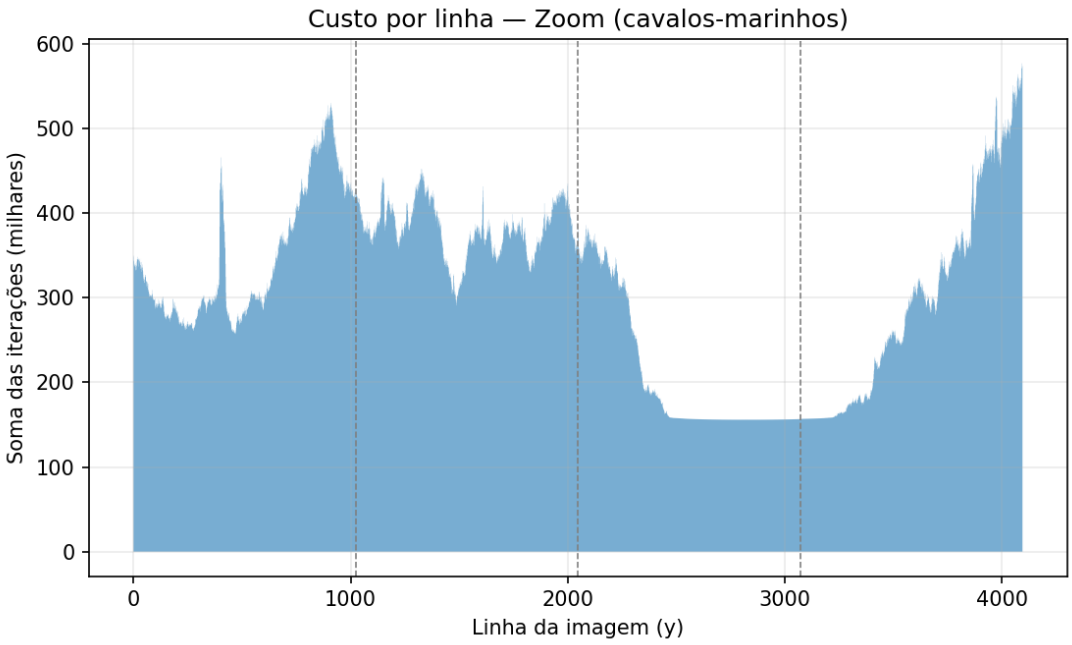


Figura 10: Distribuição do trabalho aproximado por linha no zoom.

3.4 Coleta e comunicação

O custo medido de `MPI_Gatherv` varia conforme a estratégia e a conclusão desigual dos processos. Como não há barreira entre o fim do cálculo e a coleta, os processos mais rápidos podem aguardar dentro da operação coletiva. Assim, o gráfico registra *tempo de coleta observado*, que inclui espera, e não somente transferência de bytes. Na estratégia cíclica, a montagem posterior no raiz é explicitamente contabilizada; nos blocos, a montagem adicional é nula. A escrita do arquivo binário é medida separadamente e excluída dos tempos principais de speedup.

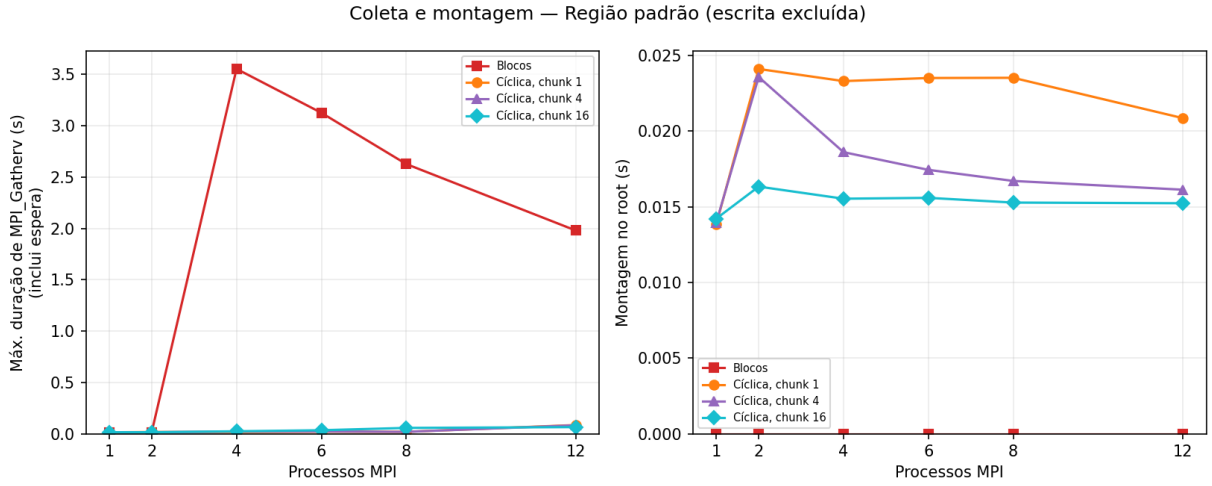


Figura 11: Tempo de `MPI_Gatherv` (com espera) e reorganização no raiz, região padrão.

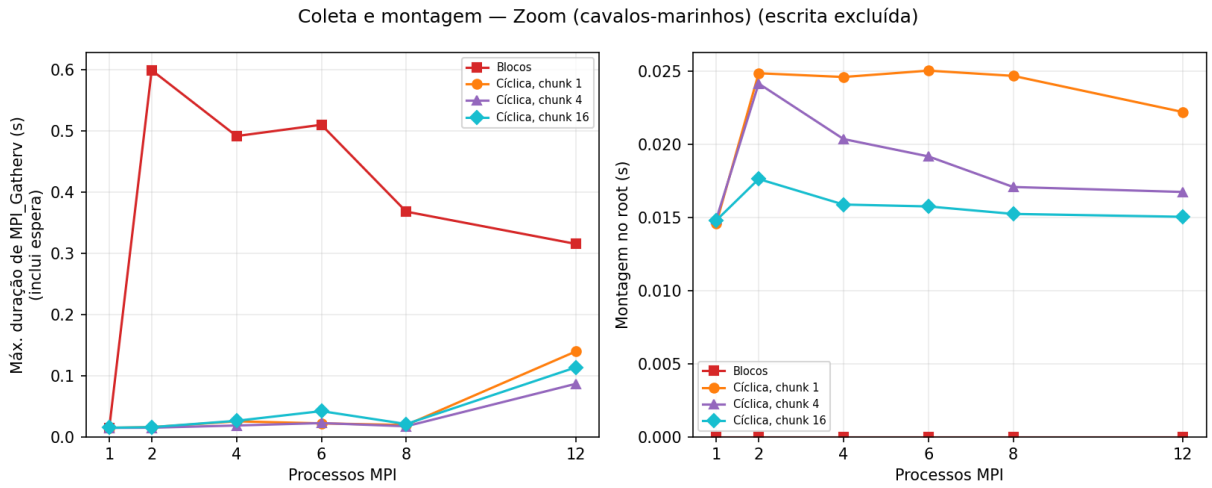


Figura 12: Coleta e reorganização na região de zoom.

3.5 Escalabilidade fraca

A escalabilidade fraca empregou os tamanhos 4096^2, 5793^2, 8192^2, 10033^2, 11585^2 e 14189^2 com 1, 2, 4, 6, 8 e 12 processos. Na maior resolução, o tempo mediano foi de **23,565 s** para blocos e **9,374 s** para cíclica com *chunk* 16. O *baseline* MPI de um processo da mesma estratégia cíclica foi de 7,463 s, resultando em eficiência fraca aproximada de **79,6%**; por blocos, o baseline foi 7,424 s e a eficiência fraca aproximada foi **31,5%**.

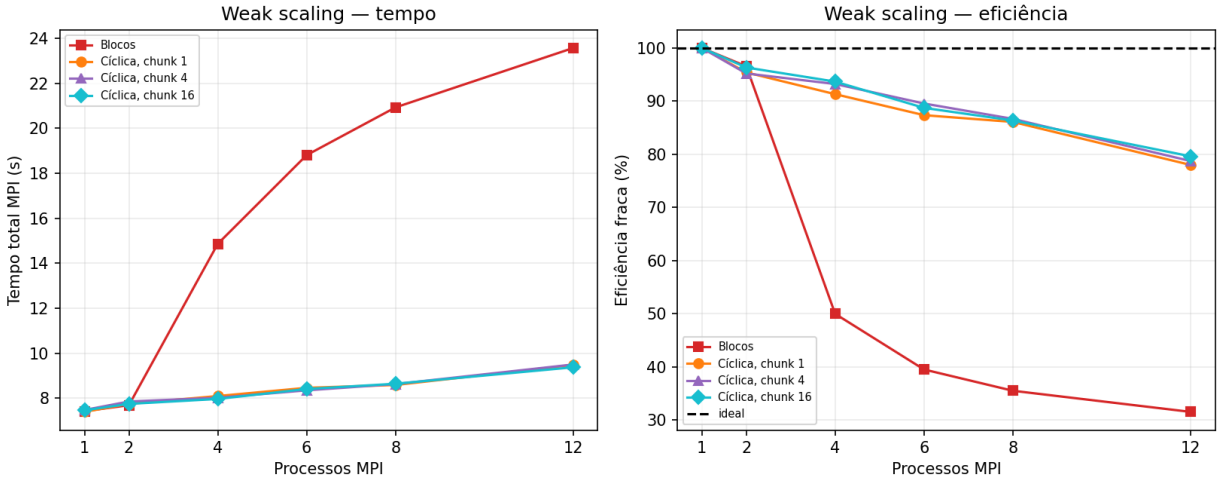


Figura 13: Escalabilidade fraca: tempo total e eficiência fraca por estratégia; trabalho nominal por processo aproximadamente constante.

3.6 Relação com a Etapa 1 (OpenMP)

A Etapa 1 avaliou *threads* OpenMP e políticas `static`, `dynamic` e `guided`; a Etapa 2 avalia processos MPI, faixas contíguas e distribuição cíclica estática. Ambas exploram independência entre pixels, mas os custos são diferentes: no OpenMP a matriz é compartilhada; no MPI é necessária a coleta explícita e, para a distribuição cíclica, reorganização no processo raiz. A comparação de tendências e

estratégias é pertinente, mas **não se devem comparar tempos absolutos como experimento controlado**, pois as coletas foram realizadas em etapas distintas e com compiladores diferentes. Por essa razão, este relatório apresenta a comparação metodológica e qualitativa entre os paradigmas, sem atribuir ganhos quantitativos de MPI sobre OpenMP que não tenham sido medidos sob um protocolo comum.

4 Discussão

A integridade MPI–sequencial foi demonstrada nas 386 execuções, mas a discrepância em relação ao gerador Python permanece uma **limitação de conformidade externa**; ela não pode ser descrita como aprovação formal. Parte da diferença se explica pela indexação i versus $i + 1$, mas o resíduo após normalização, concentrado em pixels sensíveis da dinâmica iterativa, ultrapassou os limites estabelecidos. Também não é possível atribuir a causa restante a um único fator sem investigação adicional da aritmética de ponto flutuante e das convenções de escape.

A estratégia cíclica foi superior em escalabilidade forte e fraca, principalmente na região padrão, pois distribui regiões de custos diferentes por vários processos. Os altos FBC de blocos mostram que a equivalência em número de linhas não garante equivalência em tempo de cálculo. No zoom, a vantagem da distribuição cíclica permanece, porém com diferenças relativas menores em algumas configurações.

O aumento do número de processos reduz o tempo de computação, mas não elimina os custos de criação/coordenação dos processos, coleta, montagem e sincronização. Essa observação é compatível com a lei de Amdahl: parcelas não paralelizáveis ou de coordenação limitam o ganho para tamanho fixo. Na escalabilidade fraca, a leitura de Gustafson é mais apropriada, pois cresce o problema enquanto se aumenta p; entretanto, a contagem aproximada de pixels por processo não significa exatamente o mesmo esforço iterativo em todas as regiões.

O computador contém seis núcleos físicos e doze processadores lógicos: resultados com 8 e 12 processos devem ser interpretados considerando o uso de *hyper-threading*, compartilhamento de recursos e custos do subsistema de memória. Não foi executada uma investigação isolada de afinidade, NUMA, largura de banda ou comunicação entre máquinas; assim, o experimento caracteriza **MPI local em um único host**, não desempenho de um cluster distribuído.

A estratégia cíclica requer um buffer de recepção adicional e montagem ordenada no raiz; blocos oferece composição mais simples, embora tenha apresentado desbalanceamento severo. Melhorias futuras incluem análise de afinidade, distribuição dinâmica mestre–trabalhador com filas de tarefas, redução de cópias, revisão da convenção de contagem do gerador externo e uma comparação OpenMP–MPI com condições alinhadas.

5 Conclusão

A Etapa 2 implementou e avaliou paralelismo com MPI na geração do conjunto de Mandelbrot, preservando o código sequencial da Etapa 1 e produzindo uma bateria oficial de **386 execuções válidas**. Nos testes internos, as matrizes MPI foram idênticas às sequenciais, enquanto a comparação com o gerador Python do professor identificou discrepâncias preexistentes que exigem documentação explícita.

Na região padrão, a distribuição cíclica com 12 processos atingiu **8,682 vezes** de speedup e **72,35%** de eficiência, superando blocos contíguos. No zoom, a melhor configuração destacada atingiu **7,183 vezes** de speedup. A escalabilidade fraca reforçou o efeito do balanceamento de carga: com 12

processos, a distribuição cíclica manteve aproximadamente **79,6%** de eficiência temporal relativa, contra aproximadamente **31,5%** para blocos. O principal aprendizado foi que, em trabalho irregular, a estratégia de distribuição pode ser tão determinante quanto a quantidade de processos, e que o custo de operações coletivas deve ser separado conceitualmente da espera por desbalanceamento.

6 Referências

1. UESC. *DEC107 — Processamento Paralelo: aulas de MPI e materiais da disciplina*. Material didático disponibilizado pelo professor, 2026.

2. UESC. *Projeto Conjunto de Mandelbrot — Etapa 2: enunciado e modelo de relatório*. DEC107, 2026.

3. MPI Forum. *MPI: A Message-Passing Interface Standard*. Documentação da API, operações coletivas e temporização. `https://www.mpi-forum.org/docs/`.

4. LUCAS CÉSAR, G. O.; PORTO FILHO, R. R. *Etapa 1 — Benchmark de Geração do Conjunto de Mandelbrot: sequencial e OpenMP*. Relatório da disciplina DEC107, 22 set. 2026.

5. Arquivos do experimento: `resultados_consolidados.csv`, `ambiente.md`, `validacao_referencia_professo` `analise.py` e fontes C, outubro de 2026.

Declaração de transparência e uso de inteligência artificial

Nós, **Gabriel Oliveira Lucas César** (matrícula **202410123**) e **Ronaldo Ribeiro Porto Filho** (matrícula **202410131**), estudantes do curso de Ciência da Computação, declaramos que, na elaboração do trabalho intitulado *Etapa 2 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação sequencial e paralela em memória distribuída com MPI*, submetido à disciplina DEC107 — Processamento Paralelo, utilizamos ferramentas de inteligência artificial generativa como recursos de apoio.

Opção declarada: Utilizamos ferramentas de inteligência artificial generativa como recursos de apoio.

Detalhamento do uso: ChatGPT foi utilizado como apoio ao planejamento experimental, à depuração e revisão de scripts, à organização da validação, à análise crítica dos resultados e à estruturação e revisão textual do relatório. Os programas foram executados e os resultados experimentais foram coletados e conferidos pelos autores. O detalhamento descreve o apoio com ChatGPT documentado neste projeto.

Termo de responsabilidade: Reconhecemos que ferramentas de IA podem apresentar erros e vieses. Assumimos a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido, bem como pela revisão do código, validação dos resultados e conformidade acadêmica da entrega.

Ilhéus – BA, 29 de outubro de 2026.

Gabriel Oliveira Lucas César Matrícula: 202410123	Ronaldo Ribeiro Porto Filho Matrícula: 202410131